



Containerisation and Micro-services Orchestration

Lab report – TP1 Docker

Students

Oussema BOUCHAALA
Mouin EL DABBABI
Mohamed Amin SADDOUD
Mohamed Helmi LAKHDHAR

- Part A** Graded Docker tutorial (Spring Boot, MongoDB, Docker Compose)
Part B Visit-Counter: Dockerfile, network, volume, Compose, scaling
Part C CI/CD pipeline (GitHub Actions → Docker Hub)

Contents

1	Introduction and environment	2
2	Part A – Docker tutorial (Spring Boot, MongoDB, Docker Compose)	2
2.1	Key concepts learned	3
2.2	Project 1 – <code>application-one</code> (REST API)	3
2.3	Project 2 – <code>item-service</code> with MongoDB and Docker Compose	4
3	Part B – Visit-Counter micro-services	5
3.1	Architecture	5
3.2	Part 1 – Dockerfile & isolation	6
3.3	Part 2 – Network and volume (manual deployment)	8
3.4	Part 3 – Orchestration with Docker Compose	10
3.5	Part 4 – Publication and cleaning	13
4	Reflection: RUN vs CMD	16
5	Part C – CI/CD pipeline (GitHub Actions)	16
5.1	Design	16
5.2	Results	18
6	Conclusion	19

1 Introduction and environment

The objective of this lab is to master the complete life-cycle of a container: writing an optimised and secure image, connecting containers through a user-defined network, persisting data with volumes, orchestrating several services with Docker Compose, publishing the image on Docker Hub and finally automating the whole chain with a CI/CD pipeline.

All the work was done on a Windows 11 laptop running **Docker Desktop 4.43.2** (Engine 28.3.2, WSL2 back-end). The commands were typed in the integrated terminal of Docker Desktop (PowerShell); every screenshot of this report is taken from that real session.

Project layout

```
TP_Docker_Bouchaala_ElDabbabi_Saddoud_Lakhdhar/
|-- partA-tutorial/          Part A - Spring Boot apps from the course
|   |-- application-one/     REST API + Dockerfile
|   |-- item-service-mongodb/ Spring Boot + MongoDB + docker-compose.yml
|-- partB-visit-counter/     Part B - Visit-Counter micro-services
|   |-- app/                 app.py, requirements.txt
|   |-- tests/               unit tests (pytest)
|   |-- nginx/nginx.conf     bonus load balancer
|   |-- Dockerfile .dockerignore docker-compose.yml .env
|-- .github/workflows/ci-cd.yml Part C - pipeline
'-- report/                  this LaTeX report + screenshots
```

Part 0 – Environment check



Figure 1: docker version: client and server (Docker Desktop 4.43.2, Engine 28.3.2) are available.

2 Part A – Docker tutorial (Spring Boot, MongoDB, Docker Compose)

The graded tutorial is the video course *Containerize Spring Boot CRUD App with Docker and Docker Compose* (Packt). It goes through: why containers, images vs. containers, Dockerfile, docker build, Docker Hub, running containers, inter-container communication with a MongoDB database, debugging inside a container and Docker Compose. We reproduced the two projects of the course in partA-tutorial/.

2.1 Key concepts learned

Image vs. container

An *image* is a read-only, layered template (code + runtime + libraries + configuration). A *container* is a running instance of an image with its own writable layer, process tree, network stack and file system.

Why Docker

“It works on my machine” disappears: the same image runs identically on every machine; containers start in seconds, are lighter than VMs (they share the host kernel) and are isolated from each other.

Workflow

Dockerfile $\xrightarrow{\text{docker build}}$ image $\xrightarrow{\text{docker push}}$ registry (Docker Hub) $\xrightarrow{\text{docker pull / run}}$ container.

Useful commands

`docker ps, images, logs, exec -it <c> sh, rm -f, rmi, tag, push.`

2.2 Project 1 – application-one (REST API)

A minimal Spring Boot 2.5 application exposing GET /api/v1/welcome. The course Dockerfile was FROM openjdk:11 / ADD target/app.jar / ENTRYPOINT with the jar built on the host. Because the openjdk images are deprecated and to remove the dependency on a local JDK/Maven, we wrote a **multi-stage** Dockerfile: the first stage compiles the jar with the Maven wrapper, the second stage only contains a JRE (Alpine) and the jar, and runs as a non-root user.

Listing 1: partA-tutorial/application-one/Dockerfile

```
# -----
# Part A - Tutorial: "application-one" (Spring Boot, Java 11)
# The course uses: FROM openjdk:11 / ADD target/app.jar / ENTRYPOINT
# with the jar built on the host. The openjdk images are deprecated
# and we do not want to depend on a local JDK/Maven, so we use a
# MULTI-STAGE build: the jar is compiled inside Docker (Maven wrapper)
# and only the jar is copied into a small JRE-only Alpine image.
# -----

1 # ---- Stage 1: build the jar with the Maven wrapper ----
1 FROM eclipse-temurin:11-jdk-alpine AS build
1 WORKDIR /build
1 # 1) wrapper + pom only -> dependency layer cached between builds
1 COPY .mvn .mvn
1 COPY mvnw pom.xml ./
1 RUN chmod +x mvnw && ./mvnw -q -B dependency:go-offline
1 # 2) sources (change often) -> only this layer is rebuilt
1 COPY src ./src
1 RUN ./mvnw -q -B package -DskipTests # -> target/app.jar (finalName=app)
2
2 # ---- Stage 2: runtime image (JRE only, no build tools) ----
2 FROM eclipse-temurin:11-jre-alpine
2 WORKDIR /app
2 RUN addgroup -S spring && adduser -S spring -G spring # non-root user
2 COPY --from=build /build/target/app.jar app.jar
2 USER spring
2 EXPOSE 8080
2 ENTRYPOINT ["java", "-jar", "app.jar"]
```

Both images were built on the laptop (the Maven dependencies are downloaded inside the build stage), then the REST API was tested:

```

### Part A - building the two Spring Boot images + pulling MongoDB (logs in partA-tutorial\logs)
Total time: 19.4 min

REPOSITORY          TAG          SIZE
springboot-with-mongodb  v1          293MB
application-one-tng    v1          283MB
mongo                7           1.18GB

PS C:\Users\DELL\Desktop\TP1_docker\TP_Docker_Bouchaala_ElDabbabi_Saddoud_Lakhdhar\partA-tutorial>

```

Figure 2: Part A: build of the two Spring Boot images (multi-stage) and pull of `mongo:7` – 19.4 min on our slow connection.

```

NAMES                IMAGE                STATUS                PORTS
application-one-container  application-one-tng:v1  Up About a minute    0.0.0.0:8090->8080/tcp, [::]:8090->8080/tcp
### GET http://localhost:8090/api/v1/welcome
Welcome to the world of Azure Containers!
### docker exec: user inside the container
spring
### docker logs (last lines)
2026-10-06 04:54:57.000 INFO 1 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-10-06 04:54:57.004 INFO 1 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 3 ms
PS C:\Users\DELL\Desktop\TP1_docker\TP_Docker_Bouchaala_ElDabbabi_Saddoud_Lakhdhar\partA-tutorial>

```

Figure 3: `docker run -p 8090:8080`: the endpoint `/api/v1/welcome` answers, `docker exec ...whoami` shows the non-root user `spring`, `docker logs` shows the Spring Boot logs.

As in the course, the image was also tagged and pushed to Docker Hub (`oussemabouchaala/application-one-img:v1`).

2.3 Project 2 – item-service with MongoDB and Docker Compose

A CRUD REST API (`/api/v1/items`) storing items in MongoDB. In the course the two containers are linked with the legacy `-link` option and the database is reached through `host.docker.internal:28000`. With Compose, both services share a network and the application reaches MongoDB by its service name; the connection settings of `application-local.properties` are overridden with environment variables (Spring Boot relaxed binding).

Listing 2: `partA-tutorial/item-service-mongodb/docker-compose.yml`

```

# Part A - Tutorial: run MongoDB + the Spring Boot item service together
services:
  mongodb-container-one:
    image: mongo:7
    container_name: mongodb-container-one
    ports:
      - "28000:27017" # host 28000 -> container 27017 (as in the course)
    volumes:
      - mongo-data:/data/db

  springboot-with-mongodb-container-one:
    build: .
    image: springboot-with-mongodb:v1
    container_name: springboot-with-mongodb-container-one
    ports:
      - "8090:8081" # host 8090 -> app 8081
    environment:
      # Override application-local.properties (host.docker.internal:28000):
      # inside the compose network the DB is reachable by its service name.
      SPRING_DATA_MONGODB_HOST: mongodb-container-one
      SPRING_DATA_MONGODB_PORT: "27017"
    depends_on:
      - mongodb-container-one

volumes:
  mongo-data:

```

```

v1: digest: sha256:fe2c6f7b6e97178df797fd32418c22a90dbf7f8d5845eeb86f8de6228b8ce141 size: 856
### docker compose up: MongoDB + Spring Boot item-service
Network item-service-mongodb_default Creating
Network item-service-mongodb_default Created
Volume "item-service-mongodb_mongo-data" Creating
Volume "item-service-mongodb_mongo-data" Created
Container mongodb-container-one Creating
Container mongodb-container-one Created
Container springboot-with-mongodb-container-one Creating
Container springboot-with-mongodb-container-one Created
Container mongodb-container-one Starting
Container mongodb-container-one Started
Container springboot-with-mongodb-container-one Starting
Container springboot-with-mongodb-container-one Started

NAME                                IMAGE                                STATUS                                PORTS
mongodb-container-one               mongo:7                              Up About a minute                    0.0.0.0:28000->27017/tcp, [::]:28000->27017/tcp
springboot-with-mongodb-container-one springboot-with-mongodb:v1           Up 59 seconds                        0.0.0.0:8090->8081/tcp, [::]:8090->8081/tcp

PS C:\Users\DELL\Desktop\TP1_docker\TP_Docker_Bouchaal_ElDabbabi_Saddoud_Lakhdhar\partA-tutorial\item-service-mongodb>

```

Figure 4: `docker compose up -d`: Compose creates the network, the mongo-data volume, the MongoDB container and the Spring Boot container.

```

### POST /api/v1/items (create 2 items)
201 Created -> http://localhost:8090/api/v1/items/6ac4821f3deeb43fd7da200f
201 Created -> http://localhost:8090/api/v1/items/6ac482233deeb43fd7da2010
### GET /api/v1/items
[{"_id":"6ac4821f3deeb43fd7da200f","name":"Laptop","description":"Dell XPS","price":1500,"categoryId":"c1"}, {"_id":"6ac482233deeb43fd7da2010","name":"House","description":"Wireless","price":25.5,"categoryId":"c2"}]
### the data is really in MongoDB (docker exec + mongosh)
[
  {
    _id: ObjectId('6ac4821f3deeb43fd7da200f'),
    name: 'Laptop',
    price: 1500
  },
  {
    _id: ObjectId('6ac482233deeb43fd7da2010'),
    name: 'House',
    price: 25.5
  }
]

PS C:\Users\DELL\Desktop\TP1_docker\TP_Docker_Bouchaal_ElDabbabi_Saddoud_Lakhdhar\partA-tutorial\item-service-mongodb>

```

Figure 5: End-to-end test: two POST requests return 201 Created, GET lists the items, and `docker exec ... mongosh` proves that the documents are stored in MongoDB.

3 Part B – Visit-Counter micro-services

3.1 Architecture

The application is made of two services:

- **web** – a Python/Flask application served by Gunicorn. On each request it increments the Redis key hits and answers *"Bonjour ! Cette page a été vue X fois. Je suis le conteneur [ID]"*. The container ID is the hostname of the container (Docker sets it to the short container ID).
- **db-service** – a Redis 7 (Alpine) instance with *append-only file* persistence, storing its data in the named volume `redis-data`.

The web service finds Redis by its DNS name `db-service`, resolved by the embedded DNS server of the user-defined bridge network `app-network`.

Listing 3: `app/app.py` – the web service (the HTML template, lines 21–37, is omitted)

```

import os
import socket
import time

import redis
from flask import Flask

app = Flask(__name__)

```

```

1 REDIS_HOST = os.getenv("REDIS_HOST", "db-service")
1 REDIS_PORT = int(os.getenv("REDIS_PORT", "6379"))
1
1 # The client is lazy: no connection is opened until the first command.
1 cache = redis.Redis(host=REDIS_HOST, port=REDIS_PORT, socket_connect_timeout=2)
1 def get_hit_count(client=None, retries=5):
1     """Atomically increment and return the 'hits' counter.
1
1     Retries a few times so that the app survives a restart of the
1     Redis container (stale pooled connections / DNS not yet updated).
1     """
1     client = client or cache
1     while True:
1         try:
1             return client.incr("hits")
1         except redis.exceptions.ConnectionError:
1             if retries == 0:
1                 raise
1             retries -= 1
1             time.sleep(0.5)
1
1 def container_id():
1     """Inside Docker the hostname is the short container ID."""
1     return socket.gethostname()
1
1 @app.route("/")
1 def index():
1     try:
1         count = get_hit_count()
1     except redis.exceptions.ConnectionError:
1         return (f"Database unavailable (redis://{REDIS_HOST}:{REDIS_PORT}). "
1             f"Je suis le conteneur {container_id()}"), 503
1     message = (f"Bonjour ! Cette page a été vue {count} fois. "
1         f"Je suis le conteneur {container_id()}")
1     return PAGE.format(message=message, host=REDIS_HOST)
1
1 @app.route("/health")
1 def health():
1     return {"status": "ok", "container": container_id()}

```

3.2 Part 1 – Dockerfile & isolation

Listing 4: Dockerfile (commented)

```

# =====
# Visit-Counter - Dockerfile
# Base image: python:3.12-alpine
# * Alpine Linux is ~7 MB (vs ~75 MB for Debian slim and ~350 MB
# for the full python image) -> smallest image, smallest attack
# surface, fastest push/pull.
# * Our dependencies (Flask, redis-py, gunicorn) are pure Python,
# so no compiler / musl problem: wheels install directly.
# =====
1 FROM python:3.12-alpine
1
1 # Python runtime settings: no .pyc files, unbuffered logs (docker logs)
1 ENV PYTHONDONTWRITEBYTECODE=1 \
1     PYTHONUNBUFFERED=1 \
1     APP_PORT=5000
1
1 WORKDIR /app
1
1 # Create an unprivileged user (security: never run as root)
1 RUN addgroup -S app && adduser -S -G app -H app
1
1 # ---- Docker cache optimisation ----
1 # Copy ONLY the requirements first: this layer (and the pip install
1 # below) is reused as long as requirements.txt does not change, even
1 # if the source code changes.
1 COPY app/requirements.txt .

```

```

2 RUN pip install --no-cache-dir -r requirements.txt
2
2 # Source code changes often -> copied last
3 COPY --chown=app:app app/ .
3
3 # Drop root privileges
3 USER app
3
3 EXPOSE 5000
3
3 # Production WSGI server (2 workers) instead of the Flask dev server
3 CMD ["sh", "-c", "gunicorn --workers 2 --bind 0.0.0.0:${APP_PORT} app:app"]

```

Choice of the base image. We use `python:3.12-alpine`. Alpine Linux is a minimal distribution (about 7 MB) built on `musl` and `BusyBox`. Compared with the alternatives:

Base image	Approx. size	Comment
<code>python:3.12</code> (Debian full)	≈ 1 GB	compilers, many tools – large attack surface
<code>python:3.12-slim</code> (Debian)	≈ 130 MB	good compromise, <code>glibc</code>
<code>python:3.12-alpine</code>	≈ 50–60 MB	smallest, fewer CVEs, faster pull/push

All our dependencies (Flask, `redis-py`, Gunicorn) are pure-Python wheels, so the usual drawback of Alpine (no pre-compiled `manylinux` wheels for C extensions) does not apply. The final image is **96.3 MB** as reported by Docker Desktop (uncompressed layers); the compressed image stored on Docker Hub is only **22.4 MB**.

Security. A system user `app` is created and the instruction `USER app` drops root privileges: if the application is compromised, the attacker is an unprivileged user inside the container. The application files are copied with `-chown=app:app`. A `.dockerignore` keeps tests, caches, `.env` and `.git` out of the build context.

Cache optimisation. `requirements.txt` is copied and installed *before* the source code. Docker caches each layer; as long as the dependencies do not change, the expensive `pip install` layer is reused and only the last `COPY` is rebuilt. This is visible in Figure 8: after modifying `app.py`, steps 2 to 5 are CACHED and only step 6/6 is executed.



Figure 6: First docker build `docker build -t counter-app:1.0 .` – every layer is built (base image pulled, `pip install`, ...).


```

=> => transferring context: 200B
=> CACHED [2/6] WORKDIR /app
=> CACHED [3/6] RUN addgroup -S app && adduser -S -G app -H app
=> CACHED [4/6] COPY app/requirements.txt .
=> CACHED [5/6] RUN pip install --no-cache-dir -r requirements.txt
=> CACHED [6/6] COPY --chown=app:app app/ .
=> exporting to image
=> exporting layers
=> exporting manifest sha256:2cd0af0c9bea6627e2c10ad127261dd129ee8325ca0c2b2f70e2db31867ec730
=> exporting config sha256:94ee298a4f7cd916c9d141df0772a88a6e3a7fcafc24f2568d3bd2d32b35e5
=> exporting attestation manifest sha256:3181810cb825cc30d9ccbb30ef55c8c433759163a87d5a1a75e1ed1affd6e224
=> exporting manifest list sha256:35f3309677d182f546b471359da6d9d181eca818c345aec1d8cc6eccc6a2a267
=> naming to docker.io/library/counter-app:1.0
=> unpacking to docker.io/library/counter-app:1.0

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/htknuufconrk117mcscjw178
REPOSITORY TAG IMAGE ID CREATED SIZE
counter-app 1.0 35f3309677d1 About a minute ago 96.3MB
REPOSITORY TAG IMAGE ID CREATED SIZE
PS C:\Users\DELL\Desktop\TP1_docker\TP_Docker_Bouchaal_ElDabbabi_Saddoud_Lakhdhari\partB-visit-counter>

```

Figure 7: Second build without any change: all layers come from the cache; docker images shows the size of counter-app:1.0 (96.3 MB).

```

=> [1/6] FROM docker.io/library/python:3.12-alpine@sha256:1b668429b3511ab407d8e00648891631b0b1a4d7e15e3ca70f38ab5b91ad4ab4
=> => resolve docker.io/library/python:3.12-alpine@sha256:1b668429b3511ab407d8e00648891631b0b1a4d7e15e3ca70f38ab5b91ad4ab4
=> [internal] load build context
=> => transferring context: 2.39kB
=> CACHED [2/6] WORKDIR /app
=> CACHED [3/6] RUN addgroup -S app && adduser -S -G app -H app
=> CACHED [4/6] COPY app/requirements.txt .
=> CACHED [5/6] RUN pip install --no-cache-dir -r requirements.txt
=> [6/6] COPY --chown=app:app app/ .
=> exporting to image
=> exporting layers
=> exporting manifest sha256:639a431d1a21cc2aa5d1bae89970edd5896201754a61ce4d0ca314560b51e718
=> exporting config sha256:3ecd63ae4678f4a8931bcfe31b58665f7d38a60b612788bb4aabd97435be7907
=> exporting attestation manifest sha256:a4adbffc535b0b3c8b399930149519619231cfe0a8269e54c16b968a8b19c14e
=> exporting manifest list sha256:b4f95c956ce4cf477e819bc2ce24fd6034e5a35bdc96fefdf8f448411867a56b
=> naming to docker.io/library/counter-app:1.0
=> unpacking to docker.io/library/counter-app:1.0

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/4hyk8qeonjxmns6jmfqah0sr

```

Figure 8: Build after a modification of the source code: only the layer [6/6] COPY app/ is rebuilt.

3.3 Part 2 – Network and volume (manual deployment)

A user-defined *bridge* network and a named volume are created, then both containers are started by hand:

```

docker network create app-network
docker volume create redis-data
docker run -d --name db-service --network app-network \
-v redis-data:/data redis:7-alpine redis-server --appendonly yes
docker run -d --name web-app --network app-network -p 80:5000 counter-app:1.0

```

- On a *user-defined* bridge, Docker provides an embedded DNS server (127.0.0.11): the web container reaches Redis simply with the host name db-service (the container name). This does not work on the default bridge network, where the deprecated `-link` option of the course would be needed.
- Redis is **not published** on the host (no `-p`): only containers of app-network can reach it – isolation of the database tier.
- `-p 80:5000` maps port 80 of the host to port 5000 of Unicorn.
- `--appendonly yes` makes Redis log every write to `/data/appendonly.aof`, i.e. inside the volume.

```

Unable to find image 'redis:7-alpine' locally
7-alpine: Pulling from library/redis
1633ee0c00f: Pull complete
7eb50503c2b0: Pull complete
1522f9ccf453: Pull complete
4eb14ba0e1d: Pull complete
83234986c742: Pull complete
9928009b15c8: Pull complete
4f4fb70ef34: Pull complete
Digest: sha256:858f909f9709ce576feb734aa70b8f6d624682571f9ddb6bd9a377c833b3499
Status: Downloaded newer image for redis:7-alpine
60f521830beea7ca776da290f510c8832dc9b207ae1d4c21696df4ceba190e7
6a1c4302cec36c56bac46274a383185f35dae3970b2720dd98109c57e5efb80
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAME
ES            6a1c4302cec3   counter-app:1.0        "sh -c 'gunicorn -w..." 44 seconds ago Up 7 seconds   0.0.0.0:80->5000/tcp web
-app
60f521830bee   redis:7-alpine  "docker-entrypoint.s..." About a minute ago Up About a minute 6379/tcp db-
service
PS C:\Users\DELL\Desktop\TP1_docker\TP_Docker_Bouchaal_ElDabbabi_Saddoud_Lakhidhar\partB-vlsit-counter>

```

Figure 9: Creation of app-network and redis-data, pull of redis:7-alpine and start of both containers.

```

REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
counter-app    1.0       b4f95c956ce4   35 seconds ago 96.3MB
web-app        42fd0b8f39017dde42be3e0e646469a3a3dedba6b2d4b152e3c35b3a1fa2ba0f
NAMES         IMAGE     STATUS        PORTS
web-app        counter-app:1.0 Up 8 seconds   0.0.0.0:80->5000/tcp, [::]:80->5000/tcp
db-service     redis:7-alpine Up 18 minutes 6379/tcp
PS C:\Users\DELL\Desktop\TP1_docker\TP_Docker_Bouchaal_ElDabbabi_Saddoud_Lakhidhar\partB-vlsit-counter>

```

Figure 10: docker ps: web-app publishes 0.0.0.0:80->5000/tcp, db-service only exposes 6379 internally.

```

{
  "Name": "app-network",
  "Id": "680909f93fa12a2629f227e0249facc081447e2797b31b3afab0edc8f",
  "Created": "2020-10-09T21:40:07.338658672",
  "Scope": "local",
  "Driver": "bridge",
  "EnableIPv6": true,
  "IPAM": {
    "Driver": "default",
    "Options": {},
    "Config": [
      {
        "Subnet": "172.21.0.0/16",
        "Gateway": "172.21.0.1"
      }
    ]
  },
  "Internal": false,
}

```

```

{
  "Ingress": false,
  "ConfigFrom": {
    "Network": ""
  },
  "ConfigTo": {
    "Containers": [
      {
        "Name": "web-app",
        "EndpointID": "820b094635054d66d384c01a1e9e1c0d1b5b4d3d9b968f74fa36745e",
        "MacAddress": "2a:71:5b:2b:1a:4d",
        "IPv4Address": "172.21.0.3/16",
        "IPv6Address": ""
      },
      {
        "Name": "db-service",
        "EndpointID": "f6fcbcd8c9e04d3a2c85c99a809f7b0e5dc79a080acc9d1188bf74e66",
        "MacAddress": "2a:71:5b:2b:1a:4d",
        "IPv4Address": "172.21.0.2/16",
        "IPv6Address": ""
      }
    ]
  }
}

```

Figure 11: docker network inspect app-network: bridge driver, subnet 172.21.0.0/16 (left) and the two connected containers web-app (172.21.0.3) and db-service (172.21.0.2) (right).



Figure 12: The application running in the browser on http://localhost/ – the container ID matches the one of web-app.

Robustness test

The Redis container is destroyed with `docker rm -f` and recreated with the same volume. Because the data lives in redis-data and not in the container's writable layer, the counter continues where it stopped (8 → 9). The application also retries the Redis connection a few times, so it survives the restart of its database without being restarted itself.

```

<p>Bonjour ! Cette page a été vue 7 fois. Je suis le conteneur 0829e5a8631b</p>
<p>Bonjour ! Cette page a été vue 8 fois. Je suis le conteneur 0829e5a8631b</p>

### docker rm -f db-service
db-service
web-app
### relaunch db-service with the SAME volume redis-data
5542ea38d734a4db71f7ff3fdc13d6a8e1ad498fa4ae9bbe96321a258a81daf4
### new request after restart: counter continues
<p>Bonjour ! Cette page a été vue 9 fois. Je suis le conteneur 0829e5a8631b</p>
PS C:\Users\DELL\Desktop\TP1_docker\TP_Docker_Bouchaala_Eldabbabi_Saddoud_Lakhdhar\partB-visit-counter>

```

Figure 13: Robustness test: the counter keeps its value after `docker rm -f db-service` and a relaunch on the same volume.

3.4 Part 3 – Orchestration with Docker Compose

Listing 5: `docker-compose.yml` (final version)

```

# =====
# Visit-Counter - orchestration with Docker Compose
# docker compose up -d --scale web=3
# All ports / names come from the .env file.
# =====
name: visit-counter

services:
  web:
    build: .
    image: ${IMAGE_NAME}:${IMAGE_TAG}
    # no container_name: required to be able to scale the service
    environment:
      REDIS_HOST: ${REDIS_HOST}
      REDIS_PORT: ${REDIS_PORT}
      APP_PORT: ${APP_PORT}
    ports:
      # a port RANGE so that each replica gets its own host port
      - "${WEB_HOST_PORTS}:${APP_PORT}"
    networks:
      - app-network
    depends_on:
      - db-service
    restart: unless-stopped

  db-service:
    image: redis:7-alpine
    command: ["redis-server", "--appendonly", "yes"] # AOF persistence
    volumes:
      - redis-data:/data
    networks:
      - app-network
    restart: unless-stopped

# ---- Bonus: load balancer in front of the 3 replicas ----
# started only with: docker compose --profile lb up -d --scale web=3
  lb:
    image: nginx:alpine
    profiles: ["lb"]
    ports:
      - "${LB_PORT}:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    networks:
      - app-network
    depends_on:
      - web

networks:
  app-network:
    name: app-network
    driver: bridge

volumes:
  redis-data:
    name: redis-data

```

Listing 6: .env – no port is hard-coded in the YAML

```

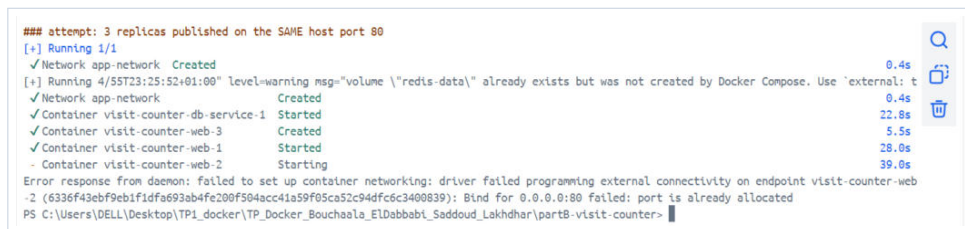
# Values used by docker-compose.yml (no hard-coded ports in the YAML)
IMAGE_NAME=counter-app
IMAGE_TAG=1.0
APP_PORT=5000
# Host port range for the scaled "web" service (3 replicas)
WEB_HOST_PORTS=8081-8083
# Host port of the load balancer (bonus)
LB_PORT=80
REDIS_HOST=db-service
1 REDIS_PORT=6379

```

The file centralises the two services, the app-network network, the redis-data volume and the environment variables. Values such as ports, image name and Redis host are read from .env through `${VARIABLE}` interpolation. The web service has **no** container_name, otherwise it could not be scaled.¹

Scaling and analysis

With a single host port ("80:5000") the command `docker compose up -d --scale web=3` fails:



```

### attempt: 3 replicas published on the SAME host port 80
[+] Running 1/1
 ✓ Network app-network Created                                0.4s
[+] Running 4/55723:25:52+01:00 level=warning msg="volume \"redis-data\" already exists but was not created by Docker Compose. Use 'external: t
 ✓ Network app-network Created                                0.4s
 ✓ Container visit-counter-db-service-1 Started              22.8s
 ✓ Container visit-counter-web-3 Created                     5.5s
 ✓ Container visit-counter-web-1 Started                     28.0s
   Container visit-counter-web-2 Starting                   39.0s
Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint visit-counter-web
-2 (6336f43ebf9eb1f1dfa693ab4fe290f504acc41a59f05ca52c94dfc6c3400839): Bind for 0.0.0.0:80 failed: port is already allocated
PS C:\Users\DELL\Desktop\TP1_docker\TP_Docker_Bouchaal_ElDabbabi_Saddoud_Lekhdhar\partB-visit-counter>

```

Figure 14: Three replicas cannot all bind port 80 of the host: *Bind for 0.0.0.0:80 failed: port is already allocated.*

Why? A TCP port of the host can be bound by **only one** process (here the docker-proxy / port-forwarder of the first replica). When the 2nd and 3rd replicas try to publish the same host port 80, the kernel refuses the bind. Containers are isolated in their own network namespaces (each one can listen on 5000 internally), but the *host* side of a port mapping is a shared, unique resource.

Solutions.

1. **Port range** (what our .env does: 8081–8083): each replica gets its own host port. It works but the client must know three addresses.
2. **Load balancer / reverse proxy** (the real solution): a single entry point (Nginx, HAProxy, Traefik) publishes port 80 and forwards each request to one of the replicas through the internal network. Docker's DNS resolves the service name `web` to the IPs of all replicas, so the proxy can distribute requests (round-robin) and the replicas need no published port at all. In production the same role is played by an orchestrator (Docker Swarm routing mesh, Kubernetes Service/Ingress) or a cloud load balancer.

¹Compose prints a warning because the volume `redis-data` was created manually in Part 2: we deliberately reuse it so the counter survives the migration from `docker run` to Compose.

```
[+] Running 5/5
  ✓ Container visit-counter-web-1      Removed      3.0s
  ✓ Container visit-counter-web-2      Removed      5.1s
  ✓ Container visit-counter-web-3      Removed      5.1s
  ✓ Container visit-counter-db-service-1 Removed      3.2s
  ✓ Network app-network                Removed      0.9s
### solution 1: one host port per replica (WEB_HOST_PORTS=8081 8083 in .env)
[+] Running 1/1
  ✓ Network app-network                Created       0.4s
[+] Running 5/5: 5/5T23:44:06+01:00" level=warning msg="volume \"/redis-data/" already exists but was not created by Docker Compose. Use 'external: t
  ✓ Network app-network                Created       0.4s
  ✓ Container visit-counter-db-service-1 Started      15.8s
  ✓ Container visit-counter-web-3      Started      33.2s
  ✓ Container visit-counter-web-1      Started      23.8s
  ✓ Container visit-counter-web-2      Started      17.0s
PS C:\Users\DELL\Desktop\TP1_docker\TP_Docker_Bouchaala_ElDabbabi_Saddoud_Lakhdhar\partB-visit-counter>
```

Figure 15: docker compose up -d -scale web=3 with the port range from .env: network, Redis and three web replicas are started.

```
NAME                IMAGE                STATUS                PORTS
visit-counter-db-service-1 redis:7-alpine       Up 39 seconds        6379/tcp
visit-counter-web-1   counter-app:1.0      Up 28 seconds        0.0.0.0:8082->5000/tcp, [::]:8082->5000/tcp
visit-counter-web-2   counter-app:1.0      Up 33 seconds        0.0.0.0:8081->5000/tcp, [::]:8081->5000/tcp
visit-counter-web-3   counter-app:1.0      Up 20 seconds        0.0.0.0:8083->5000/tcp, [::]:8083->5000/tcp
### each replica answers on its own port, same shared counter
<p>Bonjour ! Cette page a été vue 19 fois. Je suis le conteneur 84c885f319bc</p>    <- localhost:8081
<p>Bonjour ! Cette page a été vue 20 fois. Je suis le conteneur f87f55aec330</p>    <- localhost:8082
<p>Bonjour ! Cette page a été vue 21 fois. Je suis le conteneur 674dbdec10de</p>    <- localhost:8083
PS C:\Users\DELL\Desktop\TP1_docker\TP_Docker_Bouchaala_ElDabbabi_Saddoud_Lakhdhar\partB-visit-counter>
```

Figure 16: docker compose ps: replicas on ports 8081, 8082, 8083; each one answers with its own container ID while sharing the same counter.

Bonus – the load balancer implemented. We also implemented solution 2: an nginx:alpine service (Compose profile lb) publishes port 80 and proxies to web: 5000.

Listing 7: nginx/nginx.conf

```
# Bonus - round-robin load balancer in front of the "web" replicas.
# At start-up Docker's embedded DNS resolves "web" to the IPs of ALL
# replicas; nginx then distributes requests across them (round robin).
upstream visit_counter {
    zone visit_counter 64k; # shared state -> round robin across all nginx workers
    server web:5000;
}

server {
    listen 80;
    location / {
        proxy_pass http://visit_counter;
        proxy_set_header Host $host;
    }
}
```

During the first test all the requests of curl reached the same replica: nginx runs one worker per CPU and each worker kept its own round-robin pointer, so every new connection started on the first server. The zone directive puts the upstream state in shared memory; the requests are then distributed evenly (Figure 17).

```

NAME                STATUS    PORTS
visit-counter-db-service-1 Up 5 hours 6379/tcp
visit-counter-lb-1    Up 20 seconds 0.0.0.0:80->80/tcp, [::]:80->80/tcp
visit-counter-web-1    Up 5 hours 0.0.0.0:8082->5000/tcp, [::]:8082->5000/tcp
visit-counter-web-2    Up 26 seconds 0.0.0.0:8083->5000/tcp, [::]:8083->5000/tcp
visit-counter-web-3    Up 35 seconds 0.0.0.0:8081->5000/tcp, [::]:8081->5000/tcp
### 6 requests on the single entry point localhost:80 (nginx)
<p>Bonjour ! Cette page a été vue 42 fois. Je suis le conteneur f87f55aee330</p> <- request 1
<p>Bonjour ! Cette page a été vue 43 fois. Je suis le conteneur 81ce2f70f381</p> <- request 2
<p>Bonjour ! Cette page a été vue 44 fois. Je suis le conteneur 313994c9f957</p> <- request 3
<p>Bonjour ! Cette page a été vue 45 fois. Je suis le conteneur f87f55aee330</p> <- request 4
<p>Bonjour ! Cette page a été vue 46 fois. Je suis le conteneur 81ce2f70f381</p> <- request 5
<p>Bonjour ! Cette page a été vue 47 fois. Je suis le conteneur 313994c9f957</p> <- request 6
PS C:\Users\DELL\Desktop\TP1_docker\TP_Docker_Bouchaala_ELDabbabi_Saddoud_Lakhdhar\part8-visit-counter>

```

Figure 17: Single entry point `localhost:80`: the 6 requests are answered in turn by the 3 replicas (`f87f55aee330`, `81ce2f70f381`, `313994c9f957`) and the counter is shared.

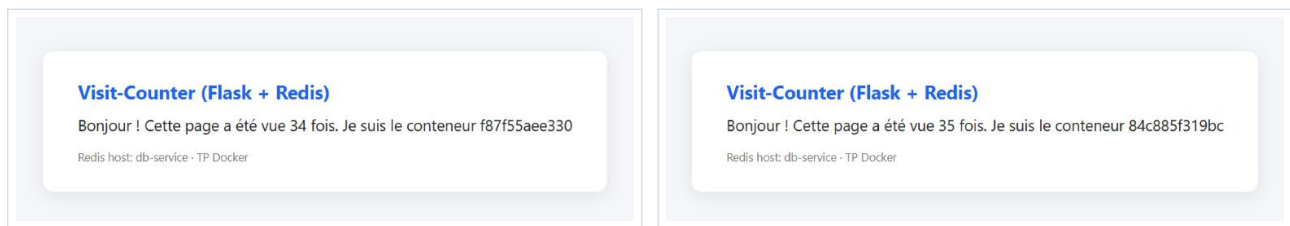


Figure 18: `docker compose -profile lb up -d --scale web=3`: two consecutive refreshes of `http://localhost/` are served by two different replicas (`f87f55aee330`, then `84c885f319bc`) – the load balancing works.

3.5 Part 4 – Publication and cleaning

Docker Hub. The image is tagged with the Docker Hub namespace (`<user>/<repository>`: `<tag>`) and pushed. Docker Desktop was signed in with our account, so the CLI was already authenticated (`docker login`).

```

docker tag counter-app:1.0 ousssemabouchaala/counter-app:1.0
docker tag counter-app:1.0 ousssemabouchaala/counter-app:latest
docker push ousssemabouchaala/counter-app:1.0
docker push ousssemabouchaala/counter-app:latest

```

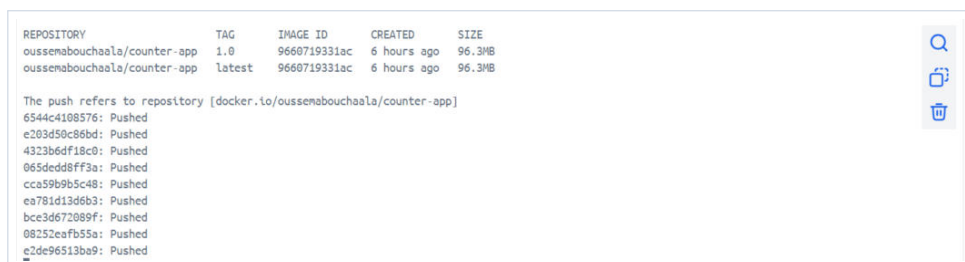


Figure 19: Tagging and pushing: every layer is uploaded to `docker.io/ousssemabouchaala/counter-app`.


```

bce3d672089f: Pushed
08252eafb55a: Pushed
e2de96513ba9: Pushed
283ac05c32e6: Pushed
1.0: digest: sha256:9660719331ac60f199e07966a65398bd819220148ff0016bc38cf810c0346d88 size: 856
The push refers to repository [docker.io/oussemabouchaala/counter-app]
065dedd8ff3a: Layer already exists
e2de96513ba9: Layer already exists
e203d50c86bd: Layer already exists
4323b6df18c0: Layer already exists
bce3d672089f: Layer already exists
283ac05c32e6: Layer already exists
ea781d13d6b3: Layer already exists
08252eafb55a: Layer already exists
6544c4108576: Layer already exists
cca59b9b5c48: Layer already exists
latest: digest: sha256:9660719331ac60f199e07966a65398bd819220148ff0016bc38cf810c0346d88 size: 856

PS C:\Users\DELL\Desktop\TP1_docker\TP_Docker_Bouchaala_Eldabbabi_Saddoud_Lakhdhar\partB-v1sit-counter>

```

Figure 20: End of the push: the second tag only re-uses the existing layers (*Layer already exists*) – both tags point to the same digest.

Docker Hub link

<https://hub.docker.com/r/oussemabouchaala/counter-app>
`docker pull oussemabouchaala/counter-app:latest`

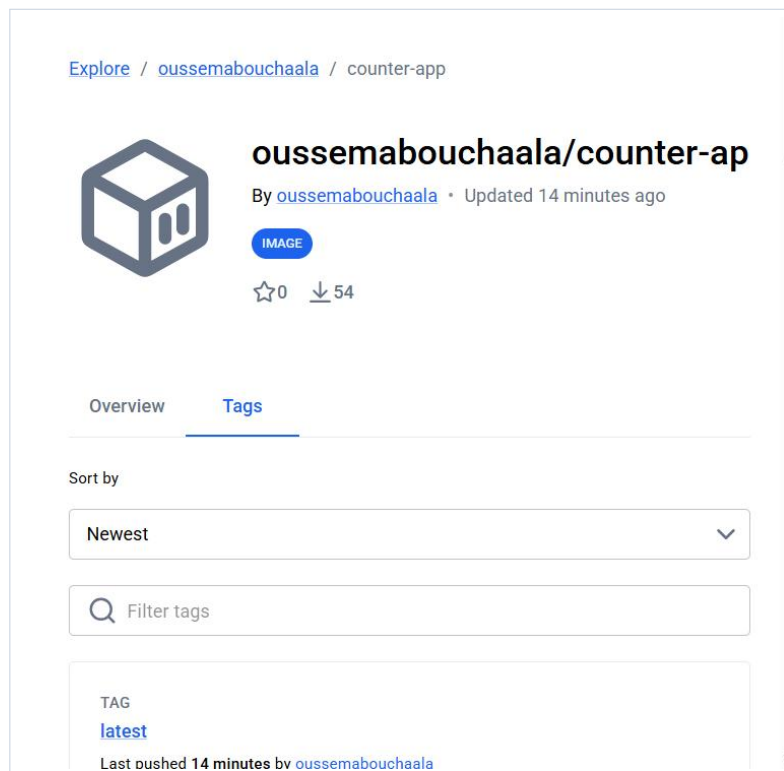


Figure 21: The public repository on Docker Hub.

Deep clean. Everything created by the project is removed with **one command**:

```
docker compose --profile lb down --rmi all --volumes --remove-orphans
```

`down` stops and deletes the containers and the network, `-rmi all` deletes the images used by the services, `-volumes` deletes the named volume and `-remove-orphans` also removes containers of services no longer in the file.

Why not `docker system prune -a -volumes`?

`docker system prune -a -volumes -f` is the “industrial” deep clean: it deletes *all* stopped containers, unused networks, *all* unused images, build cache and volumes of the whole machine. On our laptop it would also have destroyed the data of other projects (Odoo and PostgreSQL volumes...). The Compose command above is *scoped* to our project, which is the safe choice on a shared Docker host.

```
### BEFORE: resources of the Visit-Counter project
visit-counter-lb-1 Up About an hour
visit-counter-web-3 Up About an hour
visit-counter-web-2 Up About an hour
visit-counter-web-1 Up 6 hours
visit-counter-db-service-1 Up 6 hours
network: app-network
volume: redis-data

### DEEP CLEAN - one single command:
docker compose --profile lb down --rmi all --volumes --remove-orphans
Container visit-counter-lb-1 Stopping
```

Figure 22: Before: the 5 containers, the network `app-network` and the volume `redis-data` of the project.

```
Container visit-counter-web-1 Stopping
Container visit-counter-web-3 Stopping
Container visit-counter-web-2 Stopping
Container visit-counter-web-1 Stopped
Container visit-counter-web-1 Removing
Container visit-counter-web-1 Removed
Container visit-counter-web-2 Stopped
Container visit-counter-web-2 Removing
Container visit-counter-web-2 Removed
Container visit-counter-web-3 Stopped
Container visit-counter-web-3 Removing
Container visit-counter-web-3 Removed
Container visit-counter-db-service-1 Stopping
Container visit-counter-db-service-1 Stopped
Container visit-counter-db-service-1 Removing
Container visit-counter-db-service-1 Removed
Image nginx:alpine Removing
Image counter-app:1.0 Removing
Image redis:7-alpine Removing
Volume redis-data Removing
```

Figure 23: The single command stops and removes the containers, then the images (`nginx:alpine`, `counter-app:1.0`, `redis:7-alpine`), the volume and the network.

```
### AFTER the deep clean (+ removal of the local Docker Hub tags)
containers:
networks:
volumes:
images:
(empty lists = everything removed; the image stays available on Docker Hub)
PS C:\Users\DELL\Desktop\TP1_docker\TP_Docker_Bouchaala_Eldabbabi_Saddoud_Lakhdhar\partB-visit-counter>
```

Figure 24: After: no container, network, volume or image of the project is left (the image remains available on Docker Hub).

4 Reflection: RUN vs CMD

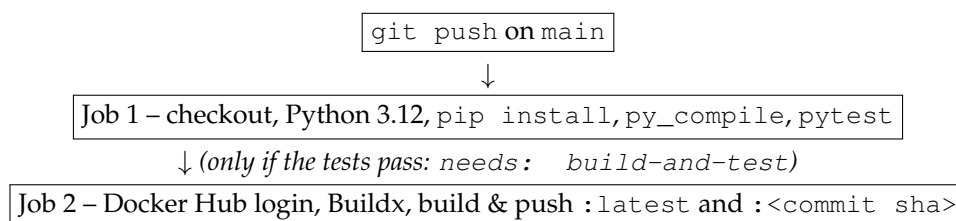
	RUN	CMD
When?	At build time (<code>docker build</code>).	At run time , each time a container starts (<code>docker run</code>).
Effect	Executes a command and commits the result as a new image layer (installed packages, created users...).	Does not create a layer: it only stores metadata – the default command of the container.
How many?	As many as needed; each one is cached.	Only the last CMD is kept.
Override	Cannot be changed when starting a container.	Overridden by the arguments of <code>docker run image <cmd></code> (or by <code>command:</code> in Compose).
Example (ours)	RUN <code>pip install -r requirements.txt</code>	CMD <code>["sh", "-c", "unicorn ...app:app"]</code>

In short: RUN *prepares* the image, CMD *says what to launch* when the image becomes a container. A related instruction is `ENTRYPOINT` (used in Part A): it defines the executable that is always run, and CMD then only provides its default arguments. The *exec form* (JSON array) is preferred because the process becomes PID 1 and receives the stop signals (`SIGTERM`) sent by `docker stop`.

5 Part C – CI/CD pipeline (GitHub Actions)

5.1 Design

The repository `OussemaBouchaala/tp-docker-visit-counter` is hosted on GitHub. We chose **GitHub Actions** instead of a self-hosted Jenkins: the runners are provided by GitHub (no server to install), the pipeline is versioned with the code (*pipeline as code*, like a `Jenkinsfile`) and the trigger on every commit is native (`on: push: branches: [main]`) – no webhook or SCM polling to configure.



- **Build and unit tests:** the 4 `pytest` tests use `fakeredis`, an in-memory Redis, so no database is needed in CI. They check the increment of the counter, the message format with the container ID, the `/health` endpoint and the HTTP 503 answer when Redis is down. A JUnit report is uploaded as an artifact.
- **Image build and publication:** executed only if the tests pass (`needs`). The credentials are never written in the code: they are stored as encrypted repository *secrets* (`DOCKERHUB_USERNAME`, `DOCKERHUB_TOKEN` – an access token, not the password). Each image is tagged with `latest` and with the commit SHA (traceability / rollback), and the Buildx layer cache (`type=gha`) speeds up the next builds.

Listing 8: .github/workflows/ci-cd.yml

```

1 name: CI-CD Visit-Counter
2
3 on:
4   push:
5     branches: [ "main" ]
6   workflow_dispatch: # manual run button (optional)
7
8 env:
9   APP_DIR: partB-visit-counter
10  IMAGE_NAME: counter-app
11
12 jobs:
13   build-and-test:
14     name: 1 - Build & unit tests
15     runs-on: ubuntu-latest
16     defaults:
17       run:
18         working-directory: ${ env.APP_DIR }
19     steps:
20       - name: Checkout code
21         uses: actions/checkout@v4
22
23       - name: Set up Python 3.12
24         uses: actions/setup-python@v5
25         with:
26           python-version: "3.12"
27           cache: pip
28           cache-dependency-path: ${ env.APP_DIR }/requirements-dev.txt
29
30       - name: Install dependencies (build)
31         run: |
32           python -m pip install --upgrade pip
33           pip install -r requirements-dev.txt
34
35       - name: Syntax check
36         run: python -m py_compile app/app.py
37
38       - name: Run unit tests
39         run: pytest -v --junitxml=test-results.xml
40
41       - name: Upload test report
42         if: always()
43         uses: actions/upload-artifact@v4
44         with:
45           name: test-results
46           path: ${ env.APP_DIR }/test-results.xml
47
48   docker-build-push:
49     name: 2 - Docker build & push to Docker Hub
50     needs: build-and-test # only if the tests passed
51     runs-on: ubuntu-latest
52     steps:
53       - name: Checkout code
54         uses: actions/checkout@v4
55
56       - name: Log in to Docker Hub
57         uses: docker/login-action@v3
58         with:
59           username: ${ secrets.DOCKERHUB_USERNAME }
60           password: ${ secrets.DOCKERHUB_TOKEN }
61
62       - name: Set up Docker Buildx
63         uses: docker/setup-buildx-action@v3
64
65       - name: Build and push image
66         uses: docker/build-push-action@v6
67         with:
68           context: ${ env.APP_DIR }
69           push: true
70           tags: |
71             ${ secrets.DOCKERHUB_USERNAME }/${ env.IMAGE_NAME }:latest
72             ${ secrets.DOCKERHUB_USERNAME }/${ env.IMAGE_NAME }:${ github.sha }
73           cache-from: type=gha
74           cache-to: type=gha,mode=max

```

Listing 9: tests/test_app.py (extract) – unit tests run by the pipeline

```

@pytest.fixture
def client(monkeypatch):
    fake = fakeredis.FakeRedis()
    monkeypatch.setattr(visit_app, "cache", fake)
    visit_app.app.config["TESTING"] = True
    with visit_app.app.test_client() as c:
        yield c

def test_counter_increments(client):
    first = client.get("/").get_data(as_text=True)
    second = client.get("/").get_data(as_text=True)
    assert "vue 1 fois" in first
    assert "vue 2 fois" in second

def test_message_contains_container_id(client, monkeypatch):
    monkeypatch.setattr(visit_app, "container_id", lambda: "abc123")
    body = client.get("/").get_data(as_text=True)

```

5.2 Results

The first run (#1) was triggered by the publication of the repository, *before* the secrets were created: job 1 (build & tests) passed, job 2 failed at *Log in to Docker Hub*. This confirms that no credential is hard-coded and that the publication job depends on the secrets. After adding them, a new commit on main (523a6db) triggered run #2 automatically: both jobs are green (tests in 9 s, image build and push in 30 s).

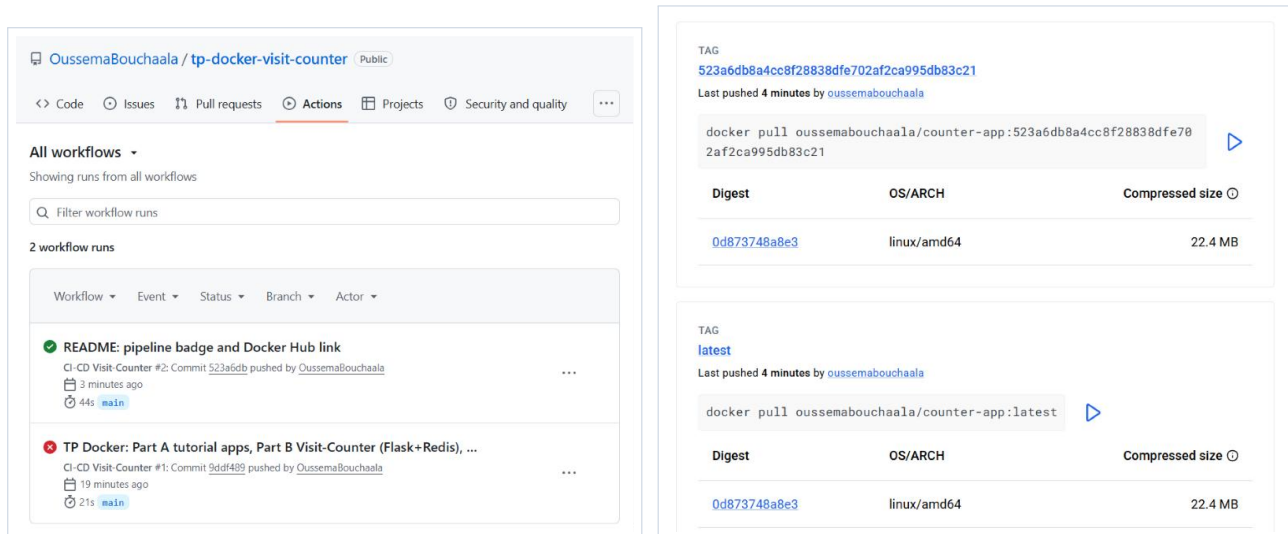


Figure 25: Left: the runs triggered by the pushes on main (#1 without secrets, #2 successful). Right: Docker Hub receives the image built by the pipeline, tagged with the commit SHA 523a6db... and latest (compressed size 22.4 MB).

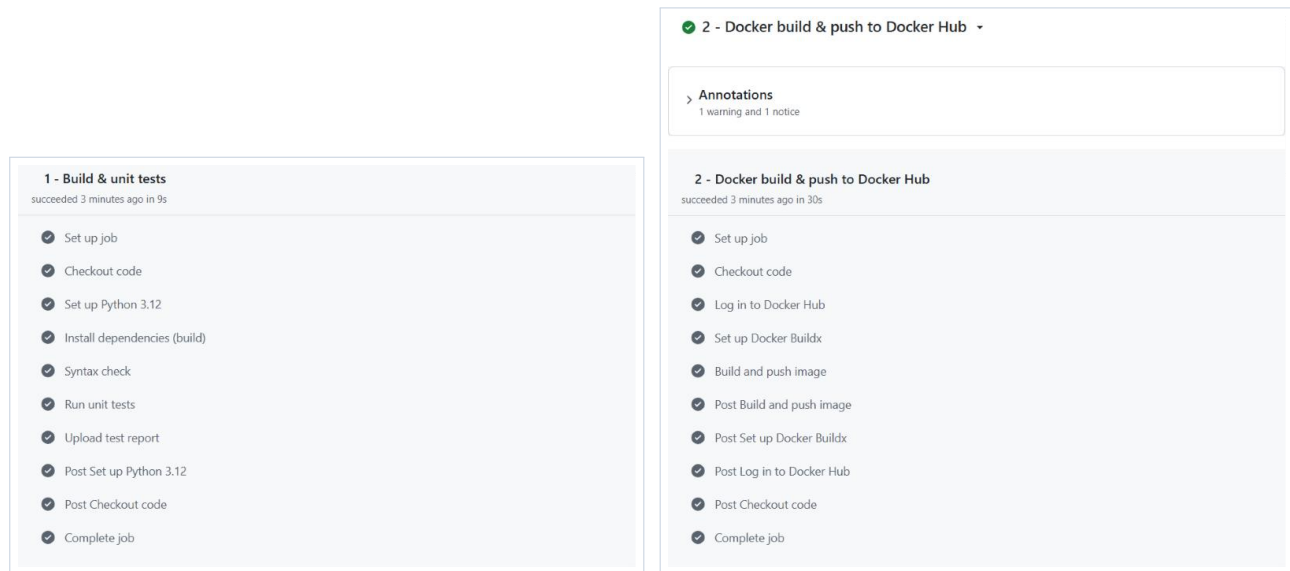


Figure 26: Run #2: job 1 (build, syntax check, unit tests, test report) and job 2 (Docker Hub login, Buildx, build and push) – all steps successful.

Links

Repository: <https://github.com/OussemaBouchaala/tp-docker-visit-counter>
Pipeline: <https://github.com/OussemaBouchaala/tp-docker-visit-counter/actions>
Image: <https://hub.docker.com/r/oussemabouchaala/counter-app>

6 Conclusion

This lab covered the complete life-cycle of a containerised application. We wrote small and secure images (Alpine base, non-root user, layer ordering for the cache, multi-stage build for Java), connected containers through a user-defined network with DNS-based service discovery, made the data survive the destruction of the container with a named volume, orchestrated and scaled the services with Docker Compose, understood why replicas cannot share a host port and solved it with a load balancer, published the image on Docker Hub, and finally automated tests, build and publication with a CI/CD pipeline triggered by every commit on `main`.